

Computer Architecture

(Computer abstractions and technology)

Dept. Applied AI, SeoulTech

Beomseok Oh

Contents

- Computer systems
- The art of managing complexity
- Number systems
- Logic gates
- CMOS transistors
- Computer architecture

What you will learn

- How programs are translated into the machine language
 - And how the hardware executes them
- The hardware/software interface
- What determines program performance
 - And how it can be improved
- How hardware designers improve performance
- What is parallel processing

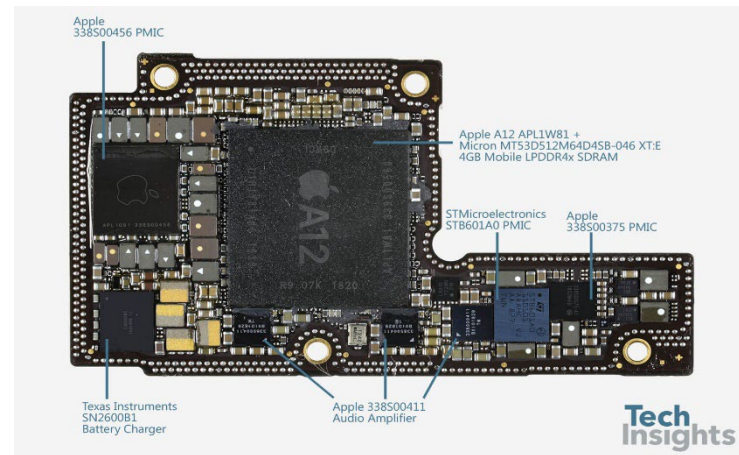
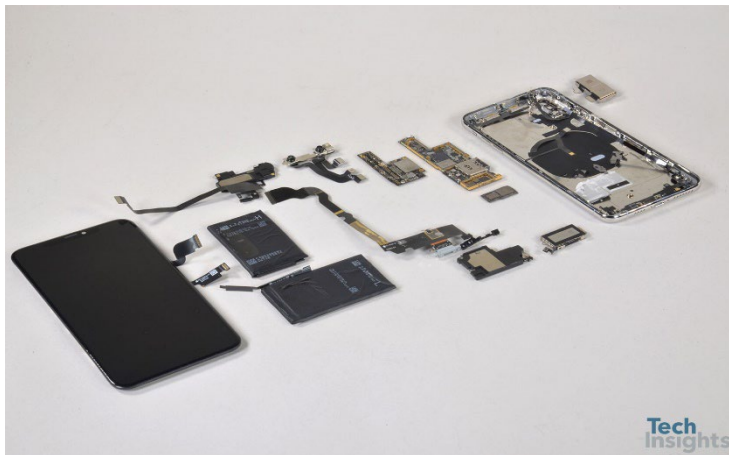
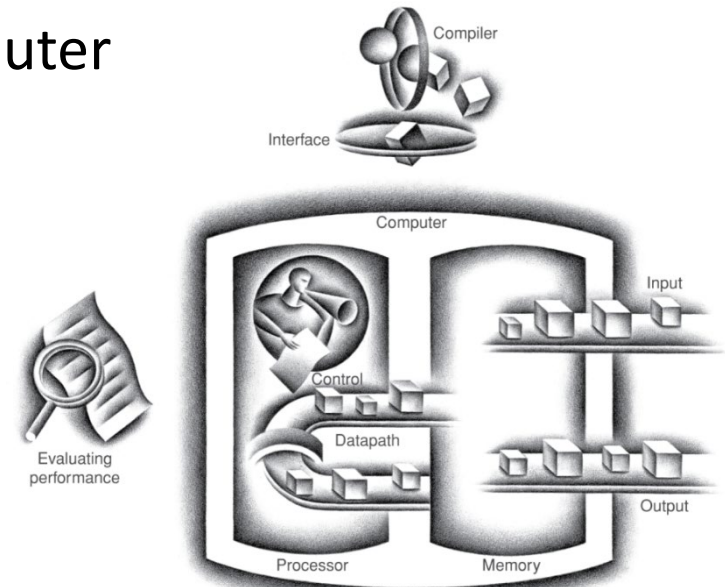
Computer systems

Classes of Computers

- Personal computers
 - General purpose, variety of software
 - Subject to cost/performance tradeoff
- Server computers
 - Network based
 - High capacity, performance, reliability
 - Range from small servers to building sized
- Supercomputers
 - Type of server
 - High-end scientific and engineering calculations
 - Highest capability but represent a small fraction of the overall computer market
- Embedded computers
 - Hidden as components of systems
 - Stringent power/performance/cost constraints

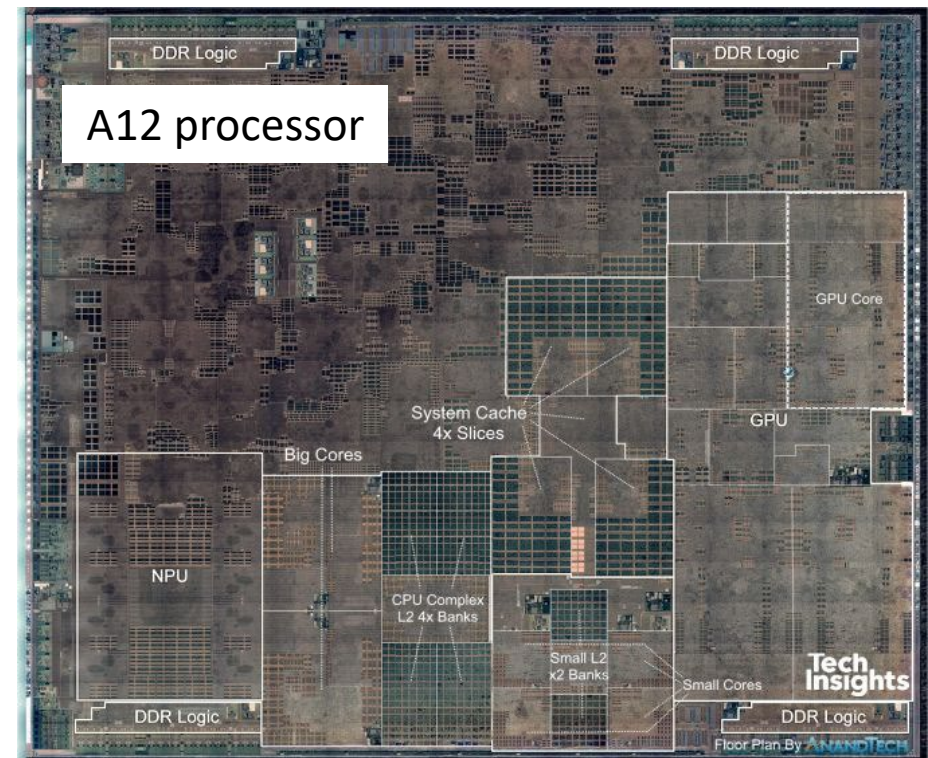
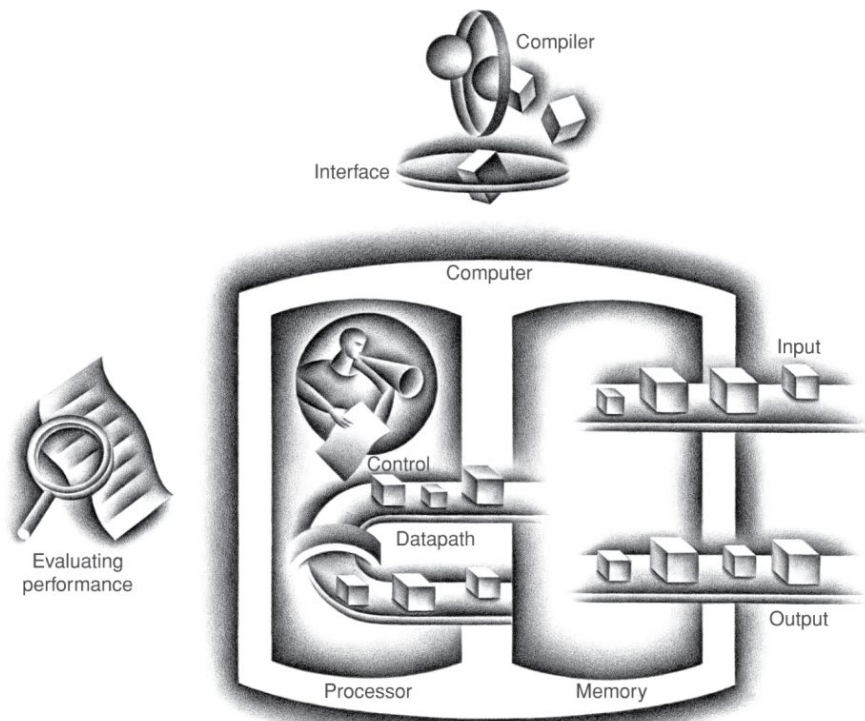
Components of a computer

- Same components for all kinds of computer
 - Desktop, server, embedded
- Input/output includes
 - User-interface devices
 - Display, keyboard, mouse
 - Storage devices
 - Hard disk, CD/DVD, flash
 - Network adapters
 - For communicating with other computers

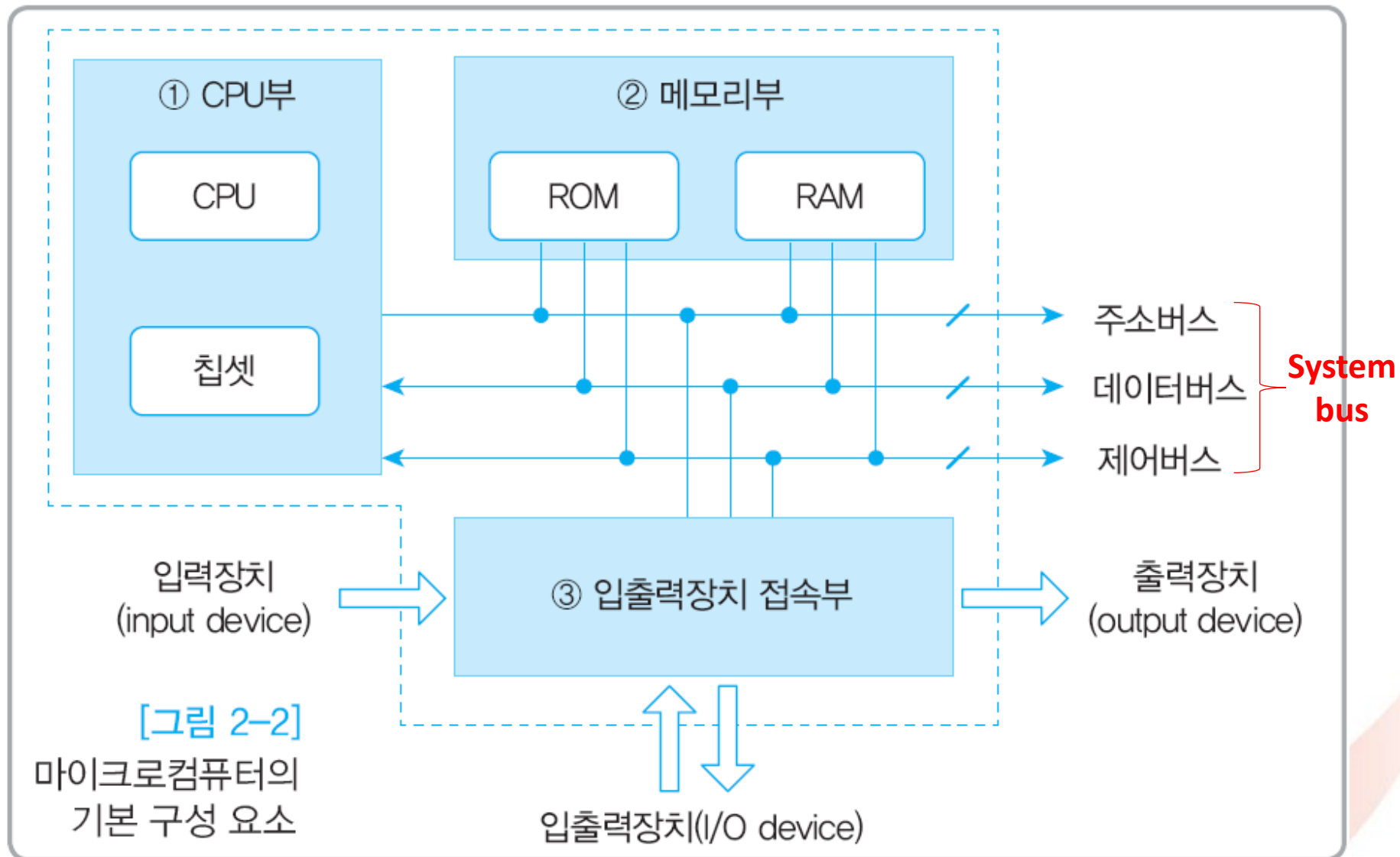


Inside the processor (CPU)

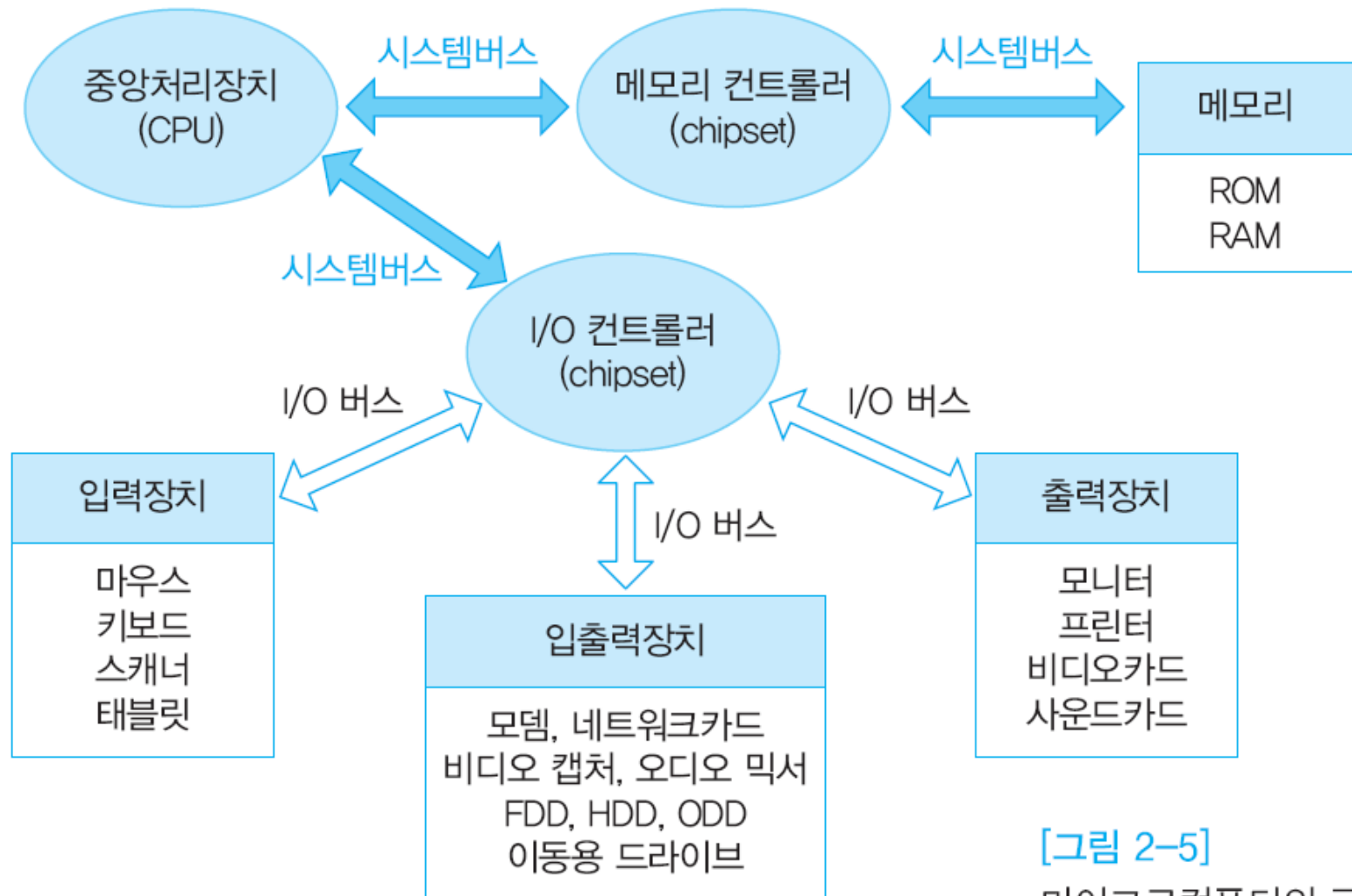
- Datapath: performs operations on data
- Control: sequences datapath, memory, ...
- Cache memory
 - Small fast SRAM memory for immediate access to data



Inside the processor (CPU) – Cont'd



Inside the processor (CPU) – Cont'd

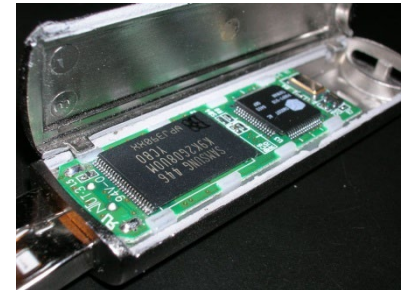


[그림 2-5]

마이크로컴퓨터의 구조

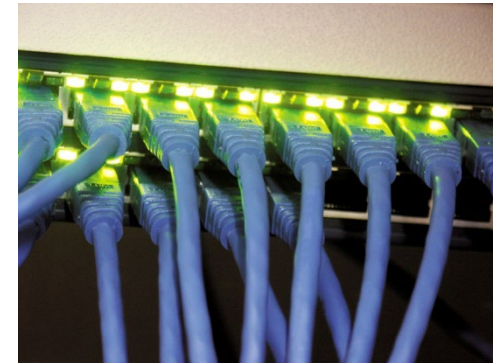
Storage devices

- Volatile main memory
 - Loses instructions and data when power off
- Non-volatile secondary memory
 - Magnetic disk
 - Flash memory
 - Optical disk (CDROM, DVD)



Networks

- Communication, resource sharing, nonlocal access
- Local area network (LAN): Ethernet
- Wide area network (WAN): the Internet
- Wireless network: WiFi, Bluetooth



The art of managing complexity

Digital Design and
Computer Architecture
RISC-V Edition



Mc

Sarah L. Harris
David Harris

The art of managing complexity

- Digital systems
 - Based on logic gates that operate on 0's and 1's
 - Modern digital systems are built from **millions or billions of transistors**
 - Human being **cannot understand** these systems by writing equations
- How do we design things that are too big to fit in one person's head at once?
 - Need to **manage complexity**
 - Abstraction
 - Discipline
 - The Three –y's
 - Hierarchy
 - Modularity
 - Regularity

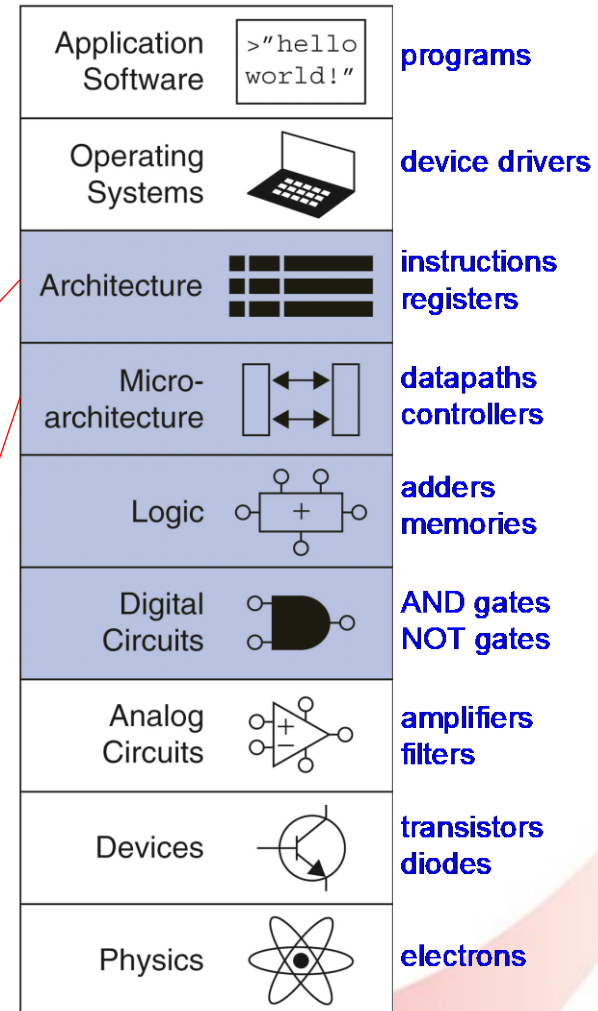
Abstraction

- Hiding details when they aren't important
 - E.g., For president elections, American politicians are interested at state level
 - E.g., Census Bureau agency consider the details of a lower level of abstraction

Describes a computer from programmer's perspective

Links the logic and architecture level (e.g., combining logic elements to execute the instructions defined by the architecture

focus of this course



Discipline

- Intentionally **restrict design choices** so that you can work more productively at a higher level of abstraction
- Example (interchangeable parts)
 - Limits the components to a standardized set with well-defined tolerances
 - Less skilled engineer could assemble and repair cars much faster
- Example: Digital discipline
 - Discrete voltages instead of continuous
 - Simpler to design than analog circuits
 - can build more sophisticated systems
 - Digital systems replacing analog predecessors
 - i.e., digital cameras, digital television, cell phones, CDs

The Three -y's

- Hierarchy
 - A system divided into modules and submodules
 - Modularity
 - Having well-defined functions and interfaces
 - Regularity
 - Encouraging uniformity, so modules can be easily reused
-
- These three principles apply to both software and hardware

Example: Model T Ford

■ Hierarchy

- Car has chassis, wheels, seats, engine
- Engine has cylinders, spark plugs, exhaust, carburetor
- Carburetor has air intake, inlet needle, feed pipe, coupling nut

■ Modularity

- Function of coupling nut
 - Hold fuel line to intake elbow
 - Prevent leaks
 - Easily removable
- Interface of coupling nut
 - Standardized diameter, thread pitch, torque

■ Regularity

- Interchangeable parts
 - Standard nut could be purchased from many suppliers

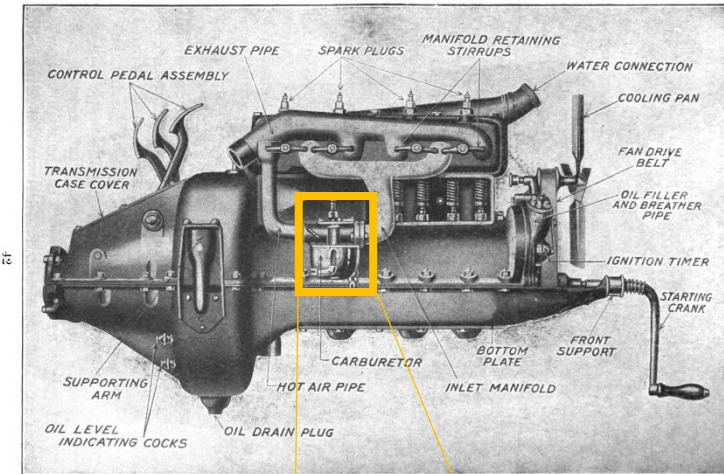


Fig. 8.—Valve Side of the Ford Model T Unit Power Plant Showing Manifolds, Carburetor and Interior of One of the Valve Spring Chambers.

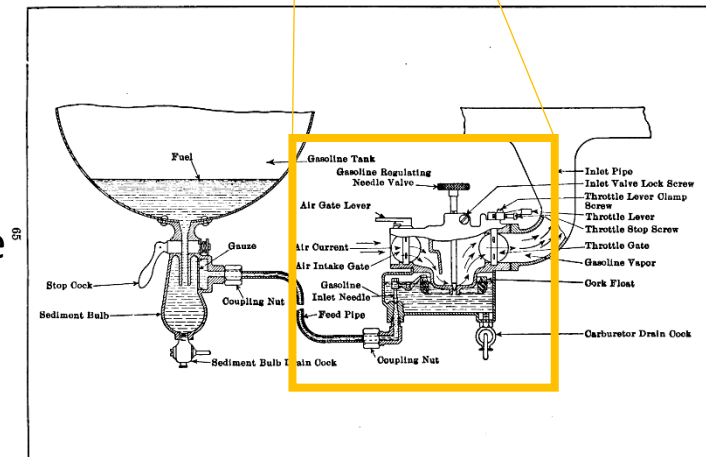


Fig. 14.—The Ford Model T Fuel Supply and Gas Making System.

The Digital Abstraction

- Most physical variables are continuous
 - Voltage on a wire
 - Frequency of an oscillation
 - Position of a mass
- Digital abstraction considers discrete subset of values

Digital Discipline: Binary Values

- Two discrete values
 - 1's and 0's
 - 1 - TRUE, HIGH
 - 0 - FALSE, LOW
- Digital circuits use voltage levels
 - 0 - low voltage (GND)
 - 1 - high voltage (V_{DD})
- The amount of information in a discrete valued variables
 - $D = \log_2 N$ bits where N distinct states
 - Bit - *Binary digit*

Number systems

Digital Design and
Computer Architecture
RISC-V Edition



Mc

Sarah L. Harris
David Harris

Number Systems

■ Decimal numbers

Decimal numbers in digital systems mean any base 10 numbers, not just those with a decimal point.

1's column
10's column
100's column
1000's column

$$5374_{10} = 5 \times 10^3 + 3 \times 10^2 + 7 \times 10^1 + 4 \times 10^0$$

five three seven four
thousands hundreds tens ones

■ Binary numbers

1's column
2's column
4's column
8's column

$$1101_2 = 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = 13_{10}$$

one one no one
eight four two one

Binary	Decimal
0	0
1	1
10	2
11	3
100	4
101	5
...	

Binary Values and Range

- N-digit decimal number
 - How many values? 10^N
 - Range? $[0, 10^N - 1]$
 - Example: 3-digit decimal number:
 - $10^3 = 1000$ possible values
 - Range: $[0, 999]$
- N-bit binary number
 - How many values? 2^N
 - Range: $[0, 2^N - 1]$
 - Example: 3-digit binary number:
 - $2^3 = 8$ possible values
 - Range: $[0, 7] = [000_2 \text{ to } 111_2]$

Hexadecimal Numbers

- Base $16 = 2^4$
- Shorthand for binary

1's column
16's column
256's column

$$2ED_{16} = 2 \times 16^2 + E \times 16^1 + D \times 16^0 = 749_{10}$$

two fourteen thirteen
two hundred fifty six's sixteens ones

- Hard to write subscripts in text files
- Some programming languages uses prefixes
 - Hex: 0x
 - $0x23AB = 23AB_{16}$
 - Binary: 0b
 - $0b1101 = 1101_2$

Hex Digit	Decimal Equivalent	Binary Equivalent
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111

Hexadecimal to Binary Conversion

- Hexadecimal to binary conversion
 - Convert $4AF_{16}$ (also written $0x4AF$) to binary
 - $0100\ 1010\ 1111_2$
- Hexadecimal to decimal conversion
 - Convert $4AF_{16}$ to decimal
 - $(16^2 \times 4) + (16^1 \times 10) + (16^0 \times 15) = 1199_{10}$

Bits, Bytes, Nibbles...

- Byte: 8 bits
 - Represents one of _____ values
- Nibble: 4 bits
 - Represents one of _____ values
- One binary digit is ___ bit
- One hex digit is _____ bits or _____ nibble
- Two hex digits make _____ byte
- Most significant on left
- Least significant on right

10010110

most significant bit least significant bit

byte

10010110

nibble

CEBF9AD7

most significant byte least significant byte

2^x vs. 10^y bytes ambiguity

Decimal term	Abbreviation	Value	Binary term	Abbreviation	Value	% Larger
kilobyte	KB	10^3	kibibyte	KiB	2^{10}	2%
megabyte	MB	10^6	mebibyte	MiB	2^{20}	5%
gigabyte	GB	10^9	gibibyte	GiB	2^{30}	7%
terabyte	TB	10^{12}	tebibyte	TiB	2^{40}	10%
petabyte	PB	10^{15}	pebibyte	PiB	2^{50}	13%
exabyte	EB	10^{18}	exbibyte	EiB	2^{60}	15%
zettabyte	ZB	10^{21}	zebibyte	ZiB	2^{70}	18%
yottabyte	YB	10^{24}	yobibyte	YiB	2^{80}	21%
ronnabyte	RB	10^{27}	robibyte	RiB	2^{90}	24%
queccabyte	QB	10^{30}	quebibyte	QiB	2^{100}	27%

- Terabyte (TB), for example
 - Originally means 1,099,511,627,776 (2^{40}) bytes
 - Communications and secondary storage systems developers started using the term to mean 1,000,000,000,000 (10^{12}) bytes

Addition and overflow

- Decimal

$$\begin{array}{r} 11 \leftarrow \text{carries} \\ 3734 \\ + 5168 \\ \hline 8902 \end{array}$$

- Binary

$$\begin{array}{r} 11 \leftarrow \text{carries} \\ 1011 \\ + 0011 \\ \hline 1110 \end{array}$$

- Overflow

- Digital systems operate on a fixed number of bits
- when result is too big to fit in the available number of bits
- Carries from the msb is discarded
- E.g., 11+6 in a 4-bit system

Signed Binary Numbers

- So far, we have considered **unsigned** binary numbers
- Several schemes exist to represent **signed** binary numbers. Among them the following two are most widely employed
 - Sign/Magnitude Numbers
 - Two's Complement Numbers

Sign/Magnitude Numbers

- 1 sign bit, N-1 magnitude bits
- Sign bit is the most significant (left-most) bit
 - Positive number: sign bit = 0
 - Negative number: sign bit = 1
- Example, 4-bit sign/mag representations of ± 6

$$+6 = 0110 \quad -6 = 1110$$

- Range of an N-bit sign/magnitude number

$$[-(2^{N-1}) + 1, 2^{N-1} - 1]$$

- Problems:

- Addition doesn't work, for example $-6 + 6$:

$$\begin{array}{r} 1110 \\ + 0110 \\ \hline 10100 \text{ (wrong!)} \end{array}$$

- Two representations of 0 (± 0):
1000, 0000

Two's Complement Numbers

- Don't have the same problems as sign/magnitude numbers
 - Addition works
 - Single representation for 0
- Two's complement numbers are **identical to unsigned binary numbers** except that the msb has weight of -2^{N-1} instead of 2^{N-1}
 - Most positive 4-bit number: 0111
 - Most negative 4-bit number: 1000
 - -1 : 1111
- The most significant bit **still indicates the sign** (i.e., sign bit)
 - 0: positive
 - 1: negative

Reversing the Sign

- Reverse the sign of a two's complement number
- Method
 1. Invert the bits
 2. Add 1 to the least significant position
- Example: Reverse the sign of $3_{10} = 0011_2$
 1. 1100
 2. $\begin{array}{r} + 1 \\ \hline 1101 = -3_{10} \end{array}$

Two's Complement Examples

- Reverse the sign of $6_{10} = 0110_2$

1. 1001

2. $+ 1$

$$1010_2 = -6_{10}$$

- What is the decimal value of the two's complement number 1001_2 ?

1. 0110

2. $+ 1$

$$0111_2 = 7_{10}, \text{ so } 1001_2 = -7_{10}$$

- Two's complement of 0? And its sign?
- Two's complement of -8?

Two's Complement Addition and subtraction

- Add $6 + (-6)$

$$\begin{array}{r} 111 \\ 0110 \\ + 1010 \\ \hline 10000 \end{array}$$

- Add $-2 + 3$

$$\begin{array}{r} 111 \\ 1110 \\ + 0011 \\ \hline 10001 \end{array}$$

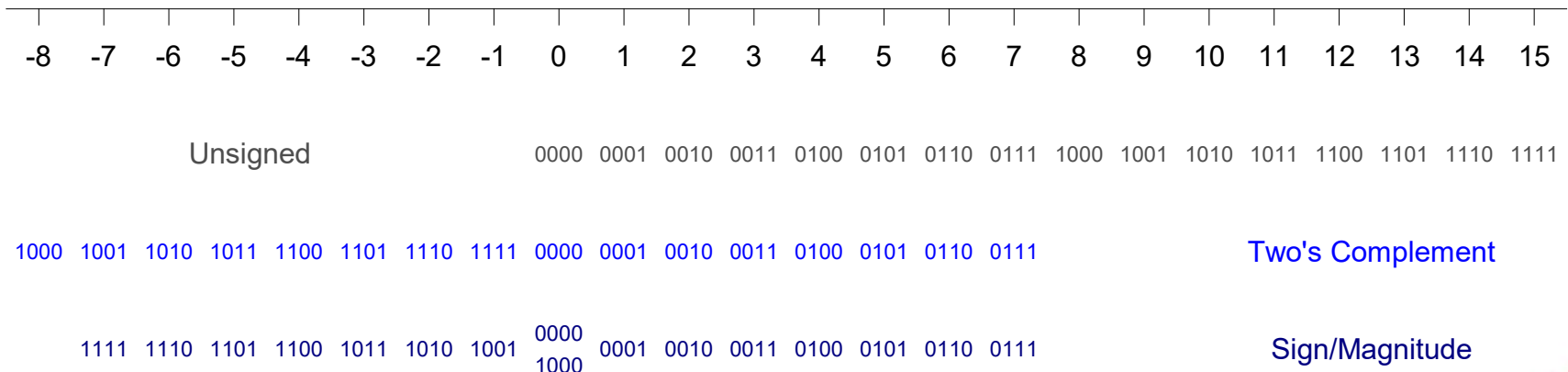
- Subtract a two's complement number by reversing the sign and adding
- Reverse sign by taking two's complement
- Ex: $3 - 5 = 3 + (-5)$

$$\begin{array}{r} 0011 \quad 3 \\ + 1011 \quad -5 \\ \hline 1110 \quad -2 \end{array}$$

Number System Comparison

Number System	Range
Unsigned	$[0, 2^N-1]$
Two's Complement	$[-2^{N-1}, 2^{N-1}-1]$
Sign/Magnitude	$[-2^{N-1}+1, 2^{N-1}-1]$

For example, 4-bit representation:



Increasing Bit Width: sign extension

- Extend number from N to M bits ($M > N$) :
 - Sign-extension for 2's complement numbers
 - Sign bit copied to msb's
 - Number value is same
 - Example 1
 - 4-bit representation of 3 = 0011
 - 8-bit sign-extended value = 00000011
 - Example 2
 - 4-bit representation of -5 = 1011
 - 8-bit sign-extended value = 11111011
 - What if we extend 0's instead of sign?
 - Example 1
 - 4-bit value = 0011 = 3_{10}
 - 8-bit zero-extended value = 00000011 = 3_{10}
 - Example 2
 - 4-bit value = 1011 = -5_{10}
 - 8-bit zero-extended value = 00001011 = 11_{10}

Logic gates

**Digital Design and
Computer Architecture**
RISC-V Edition



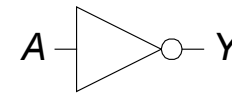
Mc

Sarah L. Harris
David Harris

Logic Gates

- Logic gates are simple digital circuits that takes one or more binary inputs and produce a binary output
- Perform logic functions
 - inversion (NOT), AND, OR, NAND, NOR, etc.
- Single-input
 - NOT gate, buffer
- Two-input
 - AND, OR, XOR, NAND, NOR, XNOR

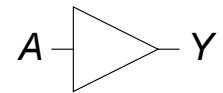
NOT



$$Y = \overline{A}$$

A	Y
0	1
1	0

BUF



$$Y = A$$

A	Y
0	0
1	1

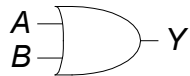
AND



$$Y = AB$$

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

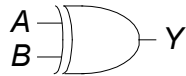
OR



$$Y = A + B$$

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

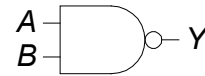
XOR



$$Y = A \oplus B$$

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

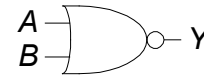
NAND



$$Y = \overline{AB}$$

A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

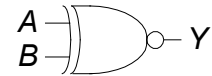
NOR



$$Y = \overline{A + B}$$

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0

XNOR



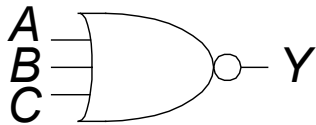
$$Y = \overline{A \oplus B}$$

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	1

Logic Gates – Cont'd

- Multiple-input

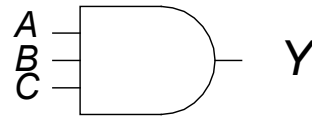
NOR3



$$Y = \overline{A+B+C}$$

A	B	C	Y
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	0

AND3



$$Y = ABC$$

A	B	C	Y
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1

Truth table rows are listed in binary order.

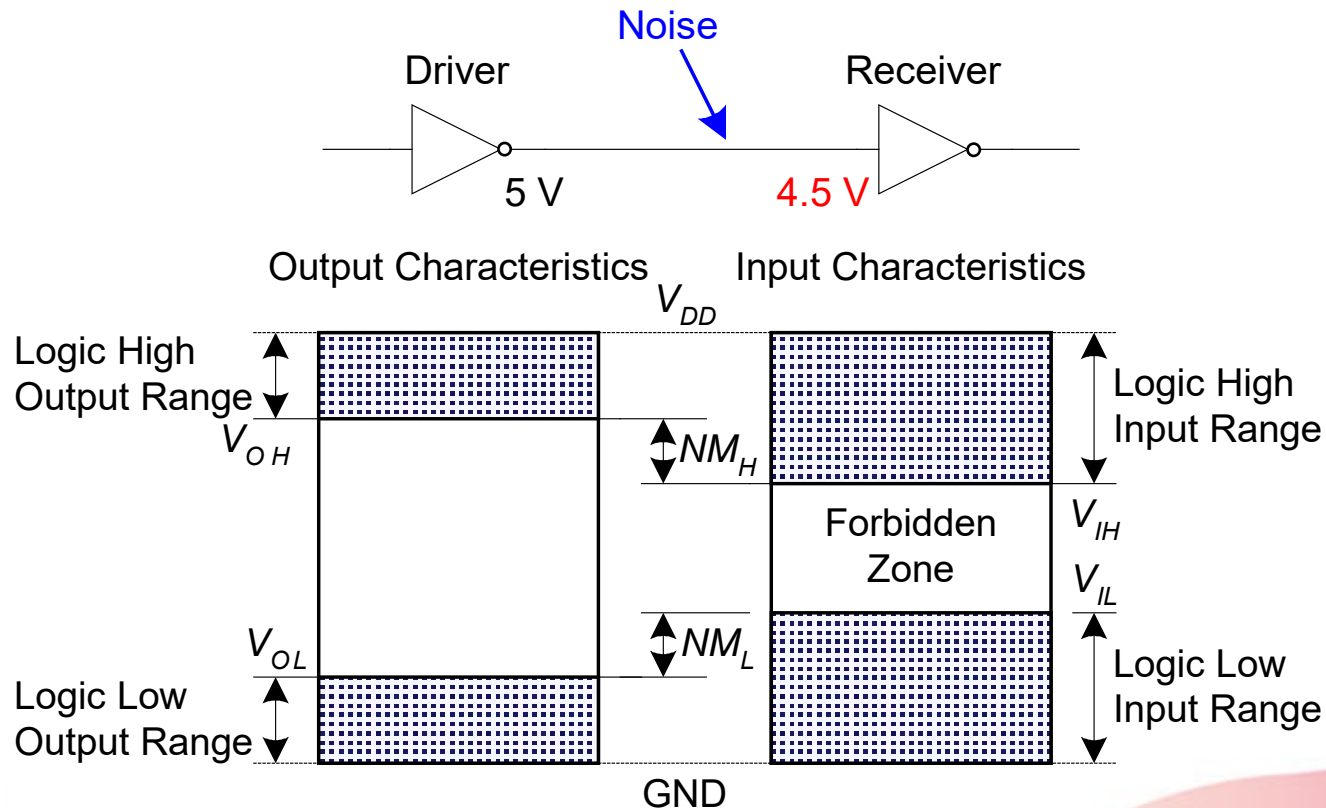
- Multi-input XOR: Odd parity

Logic Levels

- A digital system uses discrete-valued variables
 - Discrete voltages represent 1 and 0
 - For example:
 - 0 - ground (GND) or 0 volts
 - 1 - V_{DD} or 5 volts
- However, the variables are represented by **continuous physical quantities**
 - What about 4.99 volts? Is that a 0 or a 1?
 - What about 3.2 volts?
- Range of voltages for 1 and 0
- Different ranges for inputs and outputs to allow for noise

What is Noise?

- Anything that degrades the signal
 - E.g., resistance, power supply noise, coupling to neighboring wires, etc.
- Example: a gate (driver) outputs 5V but, because of resistance in a long wire, receiver gets 4.5V

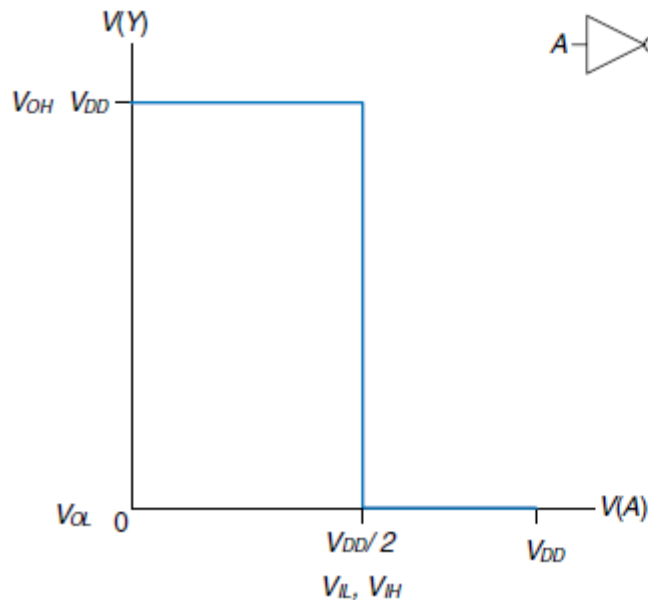


$$NM_H = V_{OH} - V_{IH}$$

$$NM_L = V_{IL} - V_{OL}$$

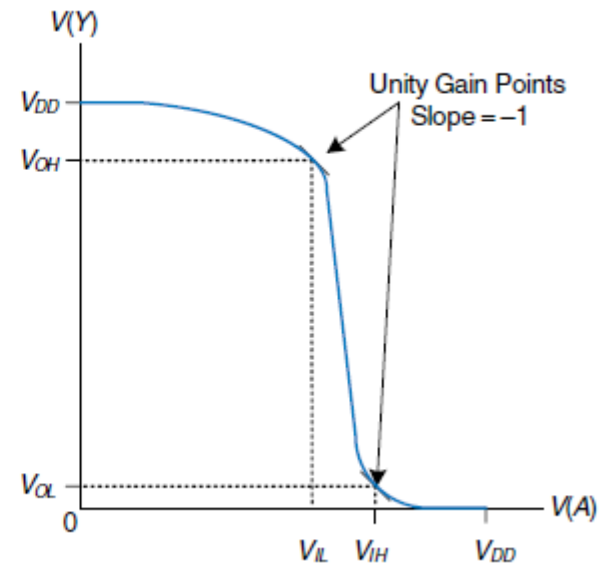
DC Transfer Characteristics

Ideal:



$$NM_H = NM_L = V_{DD}/2$$

Real:



$$NM_H, NM_L < V_{DD}/2$$

V_{DD} Scaling

- In 1970's and 1980's, $V_{DD} = 5V$
- V_{DD} has dropped
 - Avoid frying tiny transistors
 - Save power
- 3.3V, 2.5V, 1.8V, 1.5V, 1.2V, 1.0V, ...
 - Be careful connecting chips with different supply voltages



Chips operate because they contain magic smoke

- Proof: if the magic smoke is let out, the chip stops working

Logic Family	V_{DD}	V_{IL}	V_{IH}	V_{OL}	V_{OH}
TTL	5 (4.75 - 5.25)	0.8	2.0	0.4	2.4
CMOS	5 (4.5 - 6)	1.35	3.15	0.33	3.84
LVTTL	3.3 (3 - 3.6)	0.8	2.0	0.4	2.4
LVC MOS	3.3 (3 - 3.6)	0.9	1.8	0.36	2.7

CMOS transistors

Digital Design and
Computer Architecture
RISC-V Edition

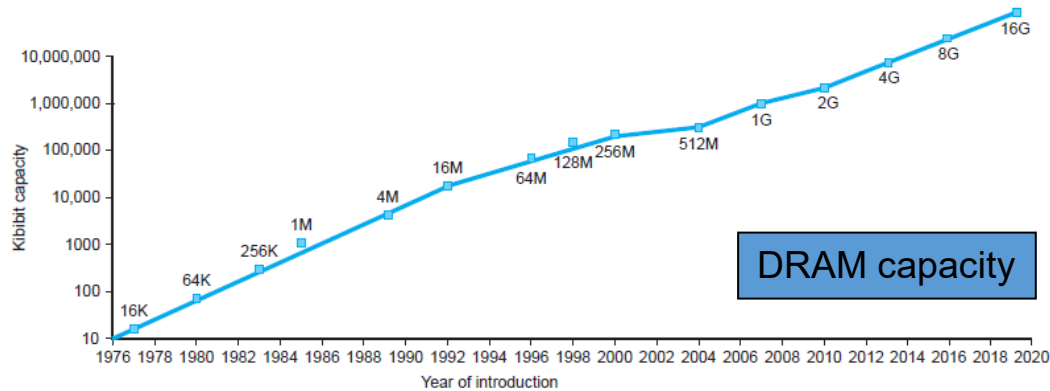


Mc

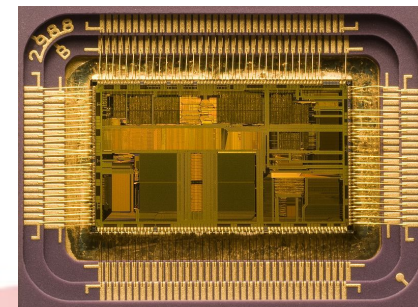
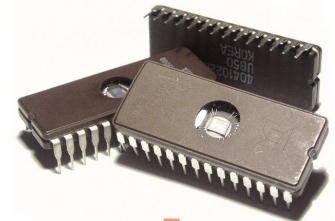
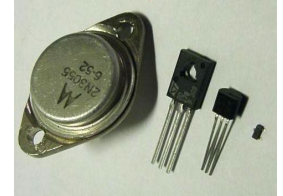
Sarah L. Harris
David Harris

Technology trends

- Electronics technology continues to evolve
 - Increased capacity and performance
 - Reduced cost

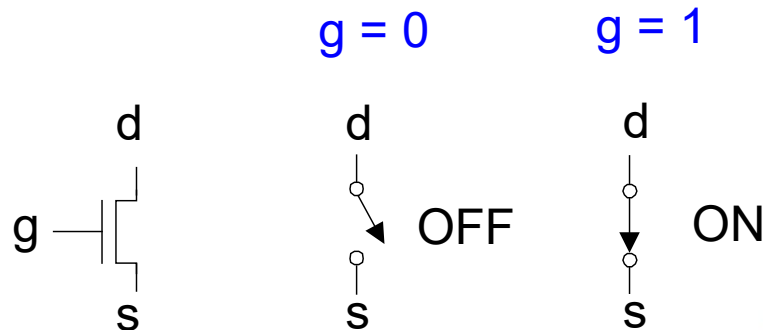


Year	Technology	Relative performance/cost
1951	Vacuum tube	1
1965	Transistor	35
1975	Integrated circuit (IC)	900
1995	Very large scale IC (VLSI)	2,400,000
2013	Ultra large scale IC (ULSI)	250,000,000,000



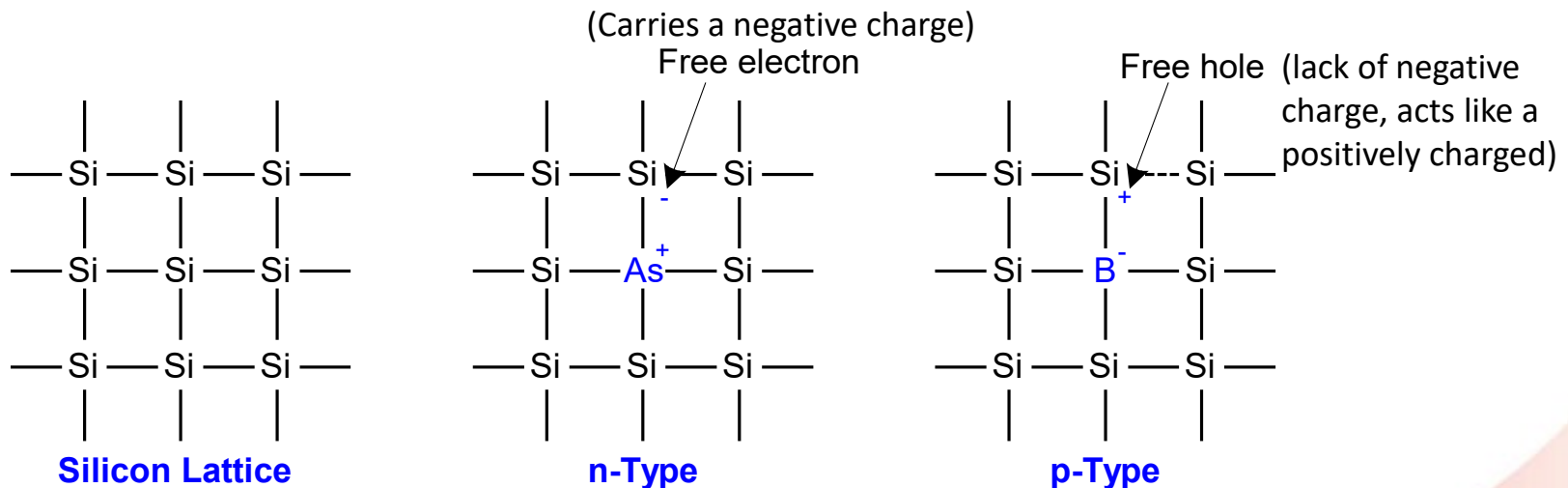
Semiconductor Technology

- Transistor
 - Electrically controlled switch, **on or off!**
 - Replaced those vacuum tubes and relays used in early computers
 - Cheap, small and reliable
- Logic gates **built from transistors**
- Metal-oxide-semiconductor field effect transistors (MOSFET or MOS transistor)
- 3-ported voltage-controlled switch
 - 2 ports connected depending on voltage of 3rd
 - d and s are connected (ON) when g is 1



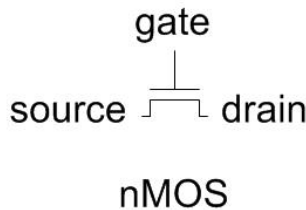
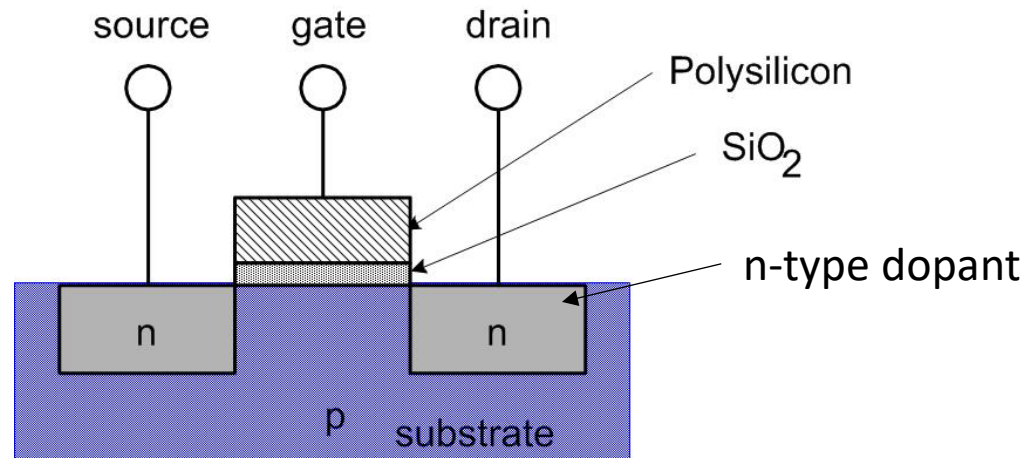
Silicon

- Transistors built from silicon, a **semiconductor**
- Pure silicon is a poor conductor (no free charges)
- Doped silicon is a good conductor (free charges)
 - When small amounts of impurities, called dopant atoms, are carefully added
 - n-type (free **negative** charges, electrons)
 - p-type (free **positive** charges, holes)



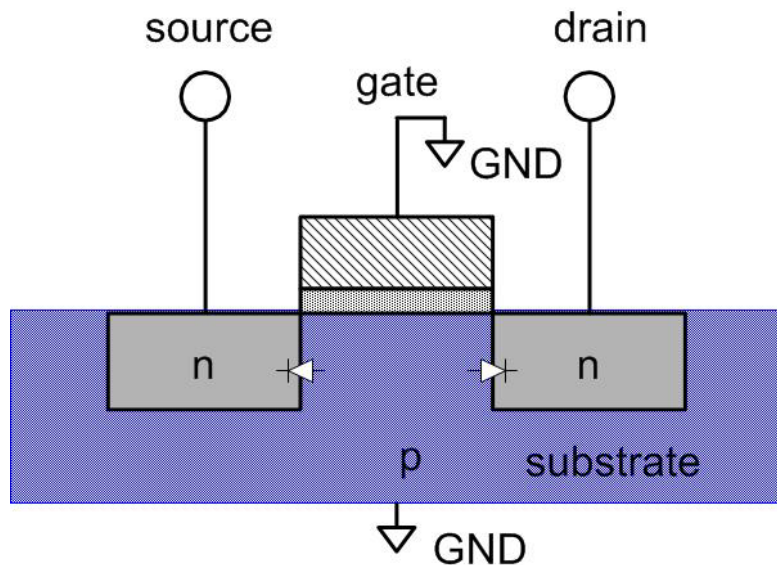
MOS Transistors

- Metal oxide silicon (MOS) transistors
 - Polysilicon (used to be metal) gate – conducting layer
 - Oxide (silicon dioxide) insulator
 - Doped silicon

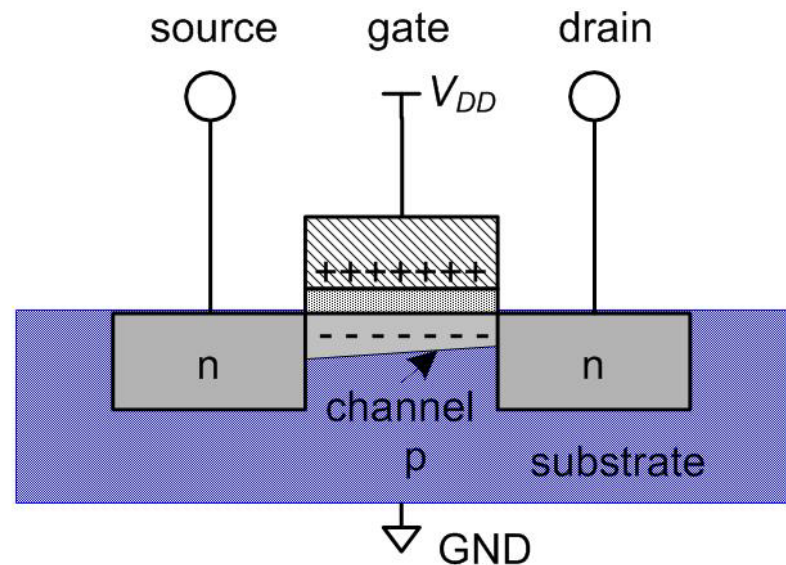


Transistors: nMOS

- Gate = 0
- OFF (no connection between source and drain)

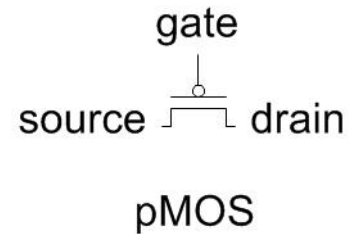
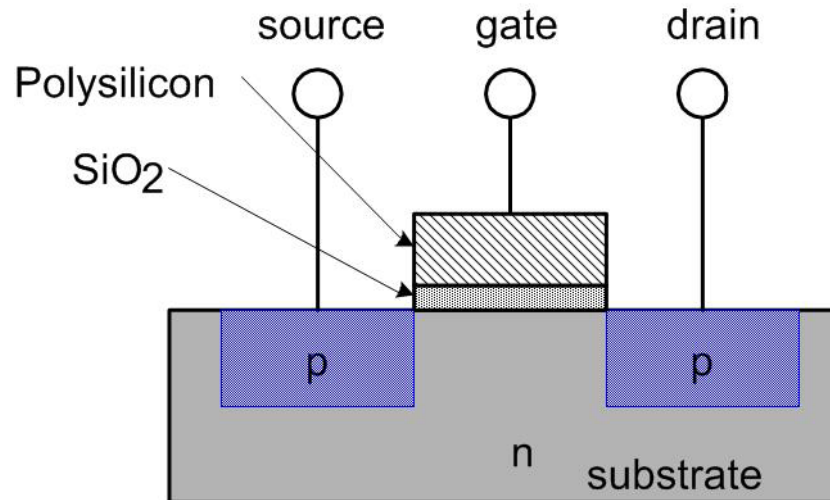


- Gate = 1
- ON (channel between source and drain)



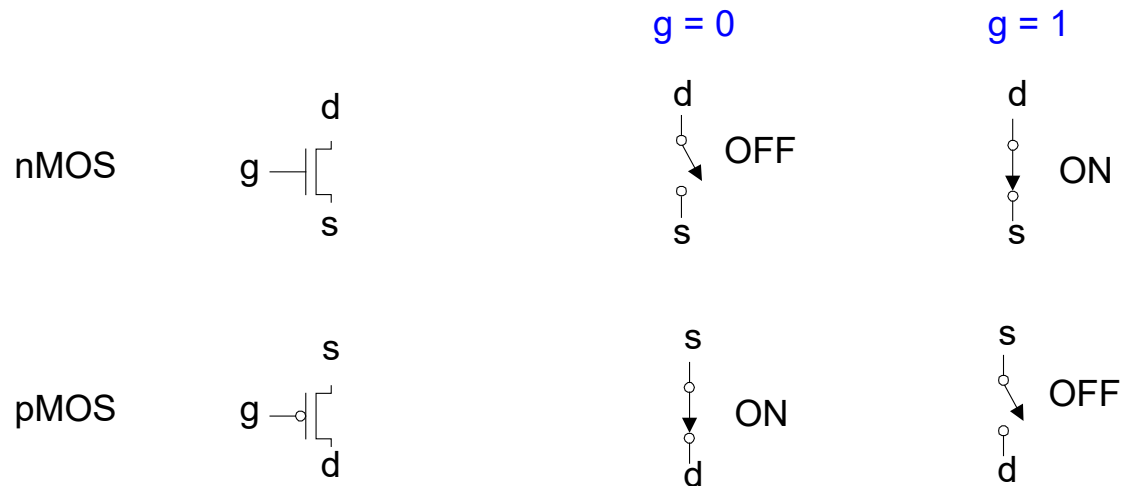
Transistors: pMOS

- pMOS transistor is opposite
 - ON when Gate = 0
 - OFF when Gate = 1

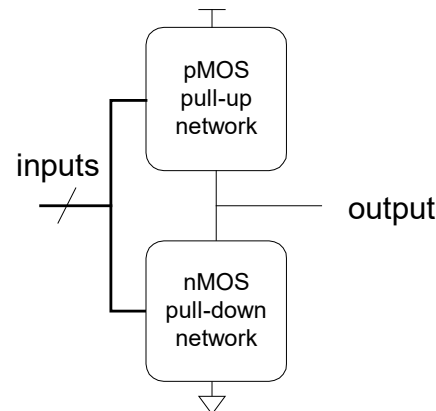


Transistor Function

- nMOS: pass 0's well, so connect source to GND
- pMOS: pass 1's well, so connect source to V_{DD}

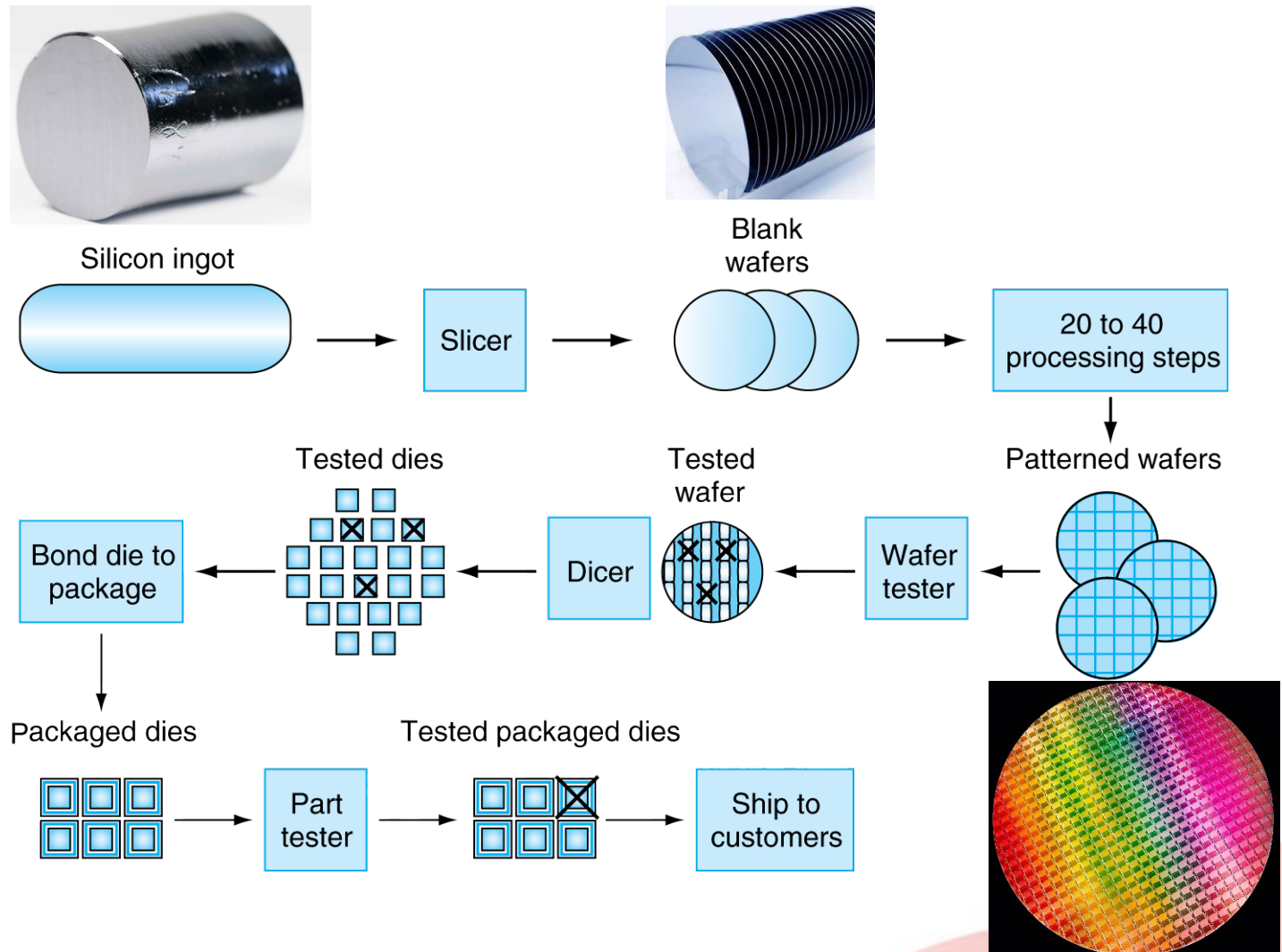


- Complement MOS (CMOS)

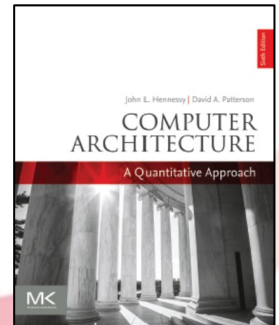


Manufacturing ICs

- Yield: proportion of working dies per wafer



Computer architecture



Seven great ideas

1. Use ***abstraction*** to simplify design
2. Make the ***common case fast***
3. Performance *via **parallelism***
4. Performance *via **pipelining***
5. Performance *via **prediction***
6. ***Hierarchy*** of memories
7. ***Dependability*** *via* redundancy



COMMON CASE FAST



ABSTRACTION



PARALLELISM



PIPELINING



PREDICTION



HIERARCHY



DEPENDABILITY

Parallelism

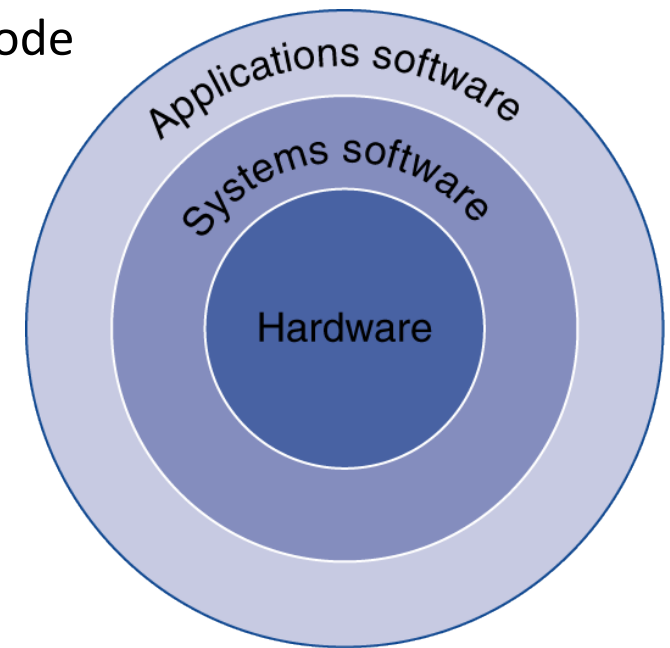
- Classes of parallelism in applications
 - Data-Level Parallelism (DLP)
 - Task-Level Parallelism (TLP)
- Classes of architectural parallelism
 - Instruction-Level Parallelism (ILP)
 - DLP
 - Pipelining, speculative execution
 - Vector architectures/Graphic Processor Units (GPUs)
 - DLP
 - A single instruction to a collection of data in parallel
 - Thread-Level Parallelism
 - Either DLP or TLP
 - Tightly coupled hardware model allowing interaction between parallel threads
 - Request-Level Parallelism
 - Parallelism among largely decoupled tasks specified by programmer or the OS

Flynn's Taxonomy

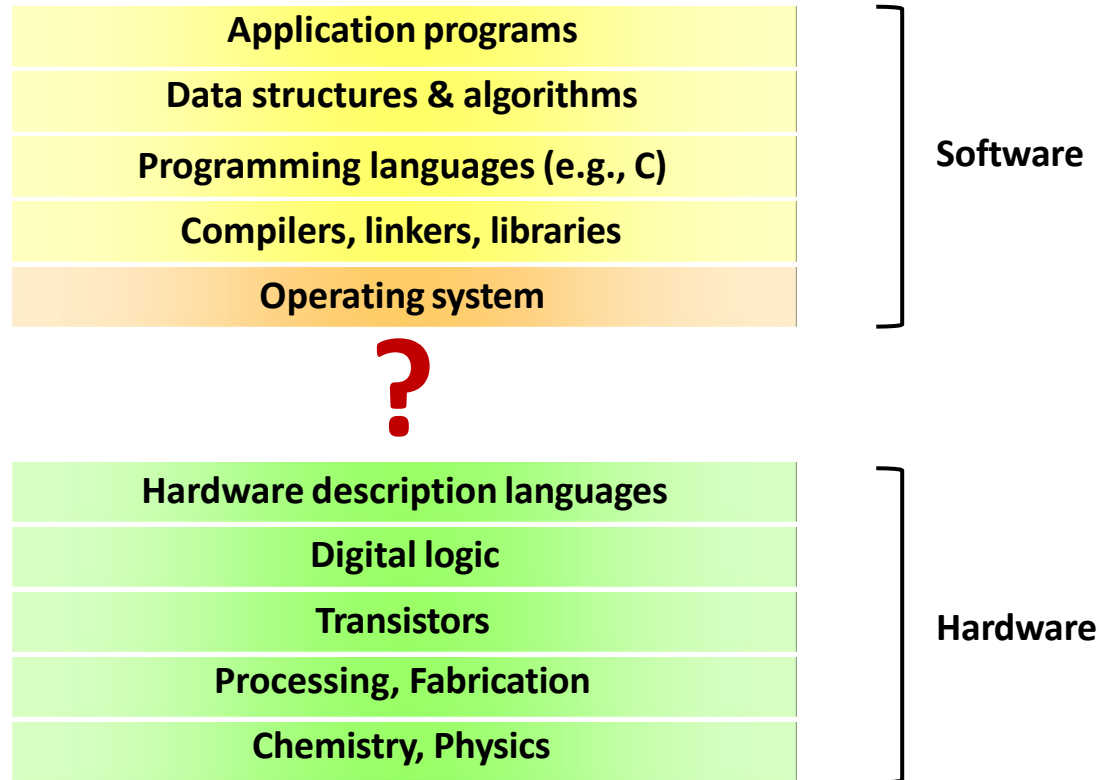
- Flynn studied the parallel computing efforts in 1960s, still being utilized
- Single instruction stream, single data stream (SISD)
 - Uniprocessor, exploits ILP
- Single instruction stream, multiple data streams (SIMD)
 - Vector architectures
 - Multimedia extensions
 - Graphics processor units
- Multiple instruction streams, single data stream (MISD)
 - No commercial implementation
- Multiple instruction streams, multiple data streams (MIMD)
 - Tightly-coupled MIMD
 - Loosely-coupled MIMD
- Many parallel processors are hybrid of the SISD, SIMD and MIMD

Software: below your program

- Application software
 - Written in high-level language
- System software
 - Compiler: translates HLL code to machine code
 - Operating system: service code
 - Handling input/output
 - Managing memory and storage
 - Scheduling tasks & sharing resources
- Hardware
 - Processor, memory, I/O controllers



How to run your program?



Level of program code

- High-level language
 - Level of abstraction closer to problem domain
 - Provides for productivity and portability

High-level
language
program
(in C)

```
swap(int v[], int k)
{int temp;
  temp = v[k];
  v[k] = v[k+1];
  v[k+1] = temp;
}
```

- Assembly language
 - Textual representation of instructions
 - For humans

Assembly
language
program
(for RISC-V)

```
swap:
  slli x6, x11, 3
  add x6, x10, x6
  ld x5, 0(x6)
  ld x7, 8(x6)
  sd x7, 0(x6)
  sd x5, 8(x6)
  jalr x0, 0(x1)
```

- Machine language
 - Hardware representation
 - Binary digits (bits)
 - Encoded instructions and data

Binary machine
language
program
(for RISC-V)

```
00000000001101011001001100010011
00000000011001010000001100110011
00000000000000110011001010000011
00000000100000110011001110000011
00000000011100110011000000100011
0000000001010011001101010000100011
00000000000000001000000001100111
```

A case for robot vacuum cleaner

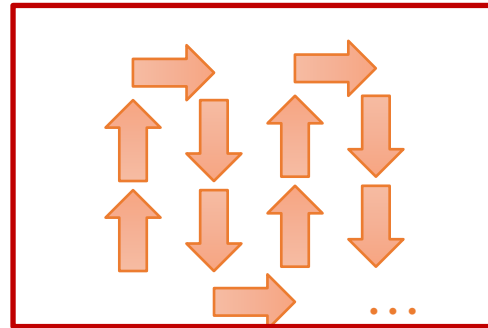
- Commands (or instructions)

- Go one step forward 00
- Go one step backward 01
- Turn left 10
- Turn right 11



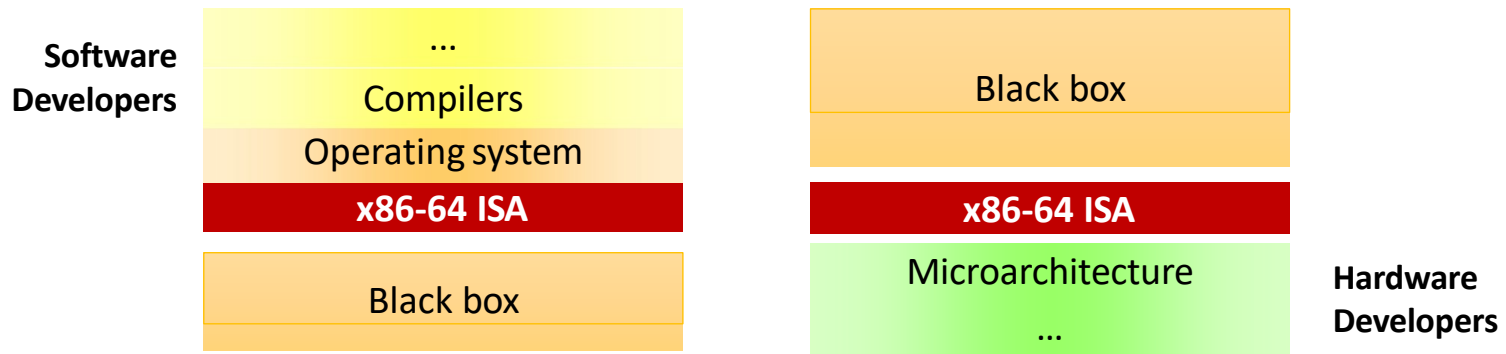
- A simple program

```
00 00 11 00 11
00 00 10 00 10
00 00 11 00 11
00 00 10 00 10 ...
```



Instruction set architecture (ISA)

- The hardware/software interface
 - Hardware abstraction visible to software (OS, compilers, ...)
 - Instructions and their encodings, registers, data types, addressing modes, etc.
 - Written documents about how the CPU behaves
 - e.g., All 64-bit Intel CPUs follow the same x86-64 (or Intel 64) ISA



Instruction Set Architecture (ISA) – Cont'd

- Class of ISA
 - General-purpose registers
 - Register-memory vs load-store
- RISC-V registers
 - 32 g.p., 32 f.p.

Register	Name	Use	Saver
x0	zero	The constant value 0	N.A.
x1	ra	Return address	Caller
x2	sp	Stack pointer	Callee
x3	gp	Global pointer	–
x4	tp	Thread pointer	–
x5-x7	t0-t2	Temporaries	Caller
x8	s0/fp	Saved register/frame pointer	Callee
x9	s1	Saved register	Callee
x10-x11	a0-a1	Function arguments/return values	Caller
x12-x17	a2-a7	Function arguments	Caller
x18-x27	s2-s11	Saved registers	Callee
x28-x31	t3-t6	Temporaries	Caller
f0-f7	ft0-ft7	FP temporaries	Caller
f8-f9	fs0-fs1	FP saved registers	Callee
f10-f11	fa0-fa1	FP function arguments/return values	Caller
f12-f17	fa2-fa7	FP function arguments	Caller
f18-f27	fs2-fs11	FP saved registers	Callee
f28-f31	ft8-ft11	FP temporaries	Caller

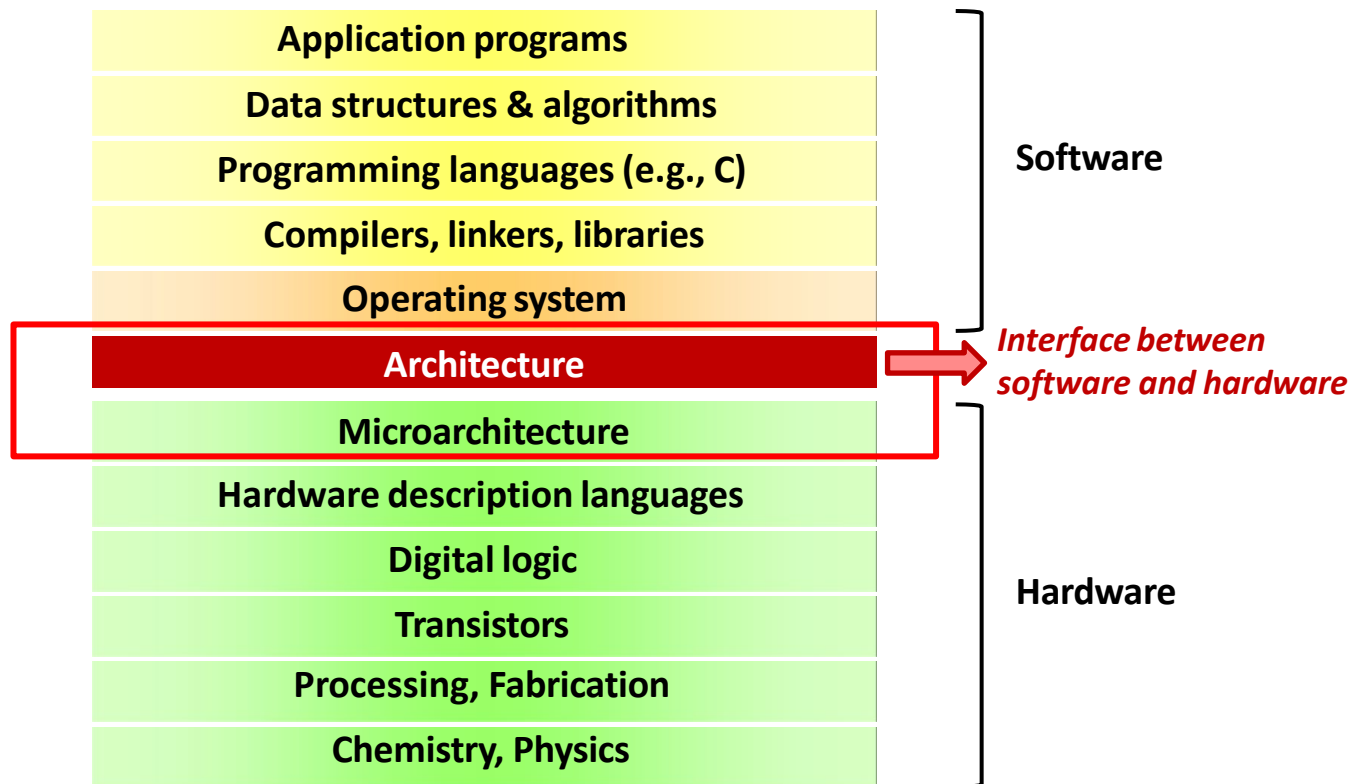
Instruction Set Architecture (ISA) – Cont'd

- Memory addressing
 - RISC-V: byte addressed, aligned accesses faster
- Addressing modes
 - RISC-V: Register, immediate, displacement (base+offset)
 - Other examples: autoincrement, indexed, PC-relative
- Types and size of operands
 - RISC-V: 8-bit, 32-bit, 64-bit
- Operations
 - RISC-V: data transfer, arithmetic, logical, control, floating point
 - See Fig. 1.5 in text
- Control flow instructions
 - Use content of registers (RISC-V) vs. status bits (x86, ARMv7, ARMv8)
 - Return address in register (RISC-V, ARMv7, ARMv8) vs. on stack (x86)
- Encoding
 - Fixed (RISC-V, ARMv7/v8 except compact instruction set) vs. variable length (x86)

Defining Computer Architecture

- “Old” view of computer architecture
 - Instruction Set Architecture (ISA) design
 - i.e. decisions regarding
 - registers, memory addressing, addressing modes, instruction operands, available operations, control flow instructions, instruction encoding
- “Real” computer architecture
 - Specific requirements of the target machine
 - Design to maximize performance within constraints: cost, power, and availability
 - Includes ISA, microarchitecture, hardware

Full levels of abstraction



Thank you!